Chapter 9. Recurrent Neural Networks and LSTM

Lectures 10/28, 11/2: RNNs, backpropagation through time, vanishing and exploding gradients, LSTM, language modeling, image captioning (naive)

Contents

1 Why Recurrent Networks **2**

2 The Recurrence **2**

3 Backpropagation Through Time **3**

3.1 The idea . . . 3

3.2 A worked two-step example . . . 3

3.3 General formulation . . . 4

4 Vanishing and Exploding Gradients **4**

4.1 RNNs are very deep networks . . . 4

4.2 Exploding gradients and gradient clipping . . . 5

4.3 Vanishing gradients . . . 5

5 Long Short-Term Memory **5**

5.1 The LSTM cell . . . 5

5.2 Gradient flow through the cell . . . 6

6 Applications **7**

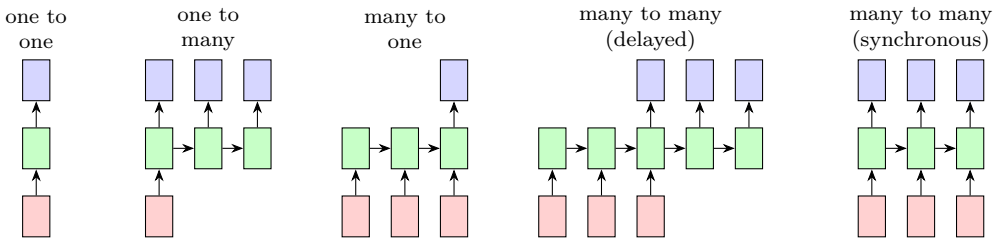
6.1 Language modeling . . . 7

6.2 Image captioning . . . 8

1 Why Recurrent Networks

For variable-length **sequences**, a recurrent network reads one element at a time and updates an **internal (hidden) state**. Patterns:

Pattern	Input → output	Examples
one to one	single → single	vanilla NN / FFN
one to many	single → sequence	image to video, image captioning
many to one	sequence → single	video classification, action recognition
many to many (delayed)	read all, then emit	translation (English to Korean), video to caption
many to many (synchronous)	emit at every step	video segmentation, video editing, style transfer



Inputs (red), states (green), outputs (blue). Delayed: inputs at $t = 1, 2, 3$, outputs at $t = 4, 5, 6$.

2 The Recurrence

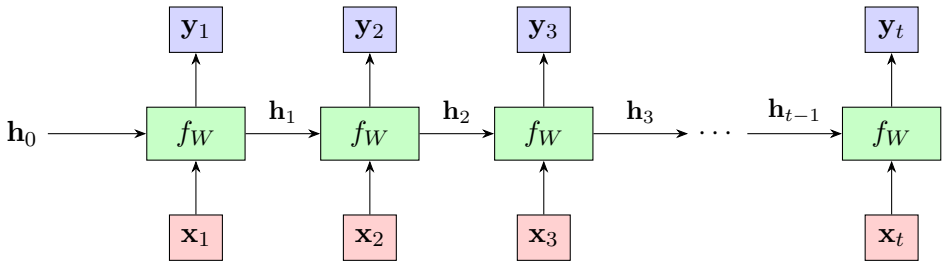
Definition 9.1: recurrent neural network

For inputs $\mathbf{x}_1, \mathbf{x}_2, \ldots,$

$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t), \qquad \mathbf{y}_t = f_{W_{hy}}(\mathbf{h}_t), \qquad t \geq 1,$$

from $\mathbf{h}_0$ (usually $\mathbf{0}$), with f_W, $f_{W_{hy}}$ and their parameters **the same at every time step**.

Unrolled, this is an ordinary graph with one parameter-sharing copy per step, so any length can be processed.



The unrolled RNN; f_W and $f_{W_{hy}}$ are shared.

Definition 9.2: vanilla RNN

With an elementwise activation σ (typically tanh),

$$\mathbf{h}_t = \sigma(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}), \qquad \hat{\mathbf{y}}_t = W^{\text{out}}\mathbf{h}_t, \qquad \mathbf{h}_0 = \mathbf{0}.$$

$$\mathbf{h}_1 = \sigma(W\mathbf{x}_1 + \mathbf{b}), \quad \hat{\mathbf{y}}_1 = W^{\text{out}}\mathbf{h}_1; \qquad \mathbf{h}_2 = \sigma(U\mathbf{h}_1 + W\mathbf{x}_2 + \mathbf{b}), \quad \hat{\mathbf{y}}_2 = W^{\text{out}}\mathbf{h}_2.$$

(The next section, following the handwritten notes, swaps the letters: W recurrent, U input.)

3 Backpropagation Through Time

3.1 The idea

$L = \sum_{t=1}^{\tau} L_t$, and L_t depends on every earlier step through the state. **Backpropagation through time (BPTT)** is backpropagation on the unrolled graph (one forward and one backward pass over the sequence); shared parameters sum the contributions of all copies.

Remark 9.3: truncated BPTT

Carry the state through the whole sequence but backpropagate only through a recent window.

3.2 A worked two-step example

Scalar RNN:

$$h_t = \tanh(Wh_{t-1} + Ux_t + b), \qquad Y_t = Vh_t + c, \qquad \theta = (W, U, V, b, c),$$

with inputs $(x_1, \dots, x_\tau)$, outputs $(Y_1, \dots, Y_\tau)$, targets $(Z_1, \dots, Z_\tau)$ and $L = \sum_t L_t$, $L_t = \frac{1}{2}(Y_t - Z_t)^2$; the chain rule acts within and across steps. Forward pass, $\tau = 2$:

$$\begin{aligned} a_1 &= Wh_0 + Ux_1 + b, \quad h_1 = \tanh a_1, \quad Y_1 = Vh_1 + c, \\ a_2 &= Wh_1 + Ux_2 + b, \quad h_2 = \tanh a_2, \quad Y_2 = Vh_2 + c, \end{aligned} \qquad L = \tfrac{1}{2}(Y_1 - Z_1)^2 + \tfrac{1}{2}(Y_2 - Z_2)^2.$$

Local derivatives. $\tanh' = 1 - \tanh^2$; ∂ is the derivative in the explicit occurrence:

$$\frac{\partial h_t}{\partial W} = (1 - h_t^2)h_{t-1}, \qquad \frac{\partial h_t}{\partial U} = (1 - h_t^2)x_t, \qquad \frac{\partial h_t}{\partial b} = 1 - h_t^2, \qquad \frac{\partial h_2}{\partial h_1} = (1 - h_2^2)W.$$

Through Y_t,

$$\frac{\partial L_t}{\partial h_t} = \frac{\partial L_t}{\partial Y_t}\frac{\partial Y_t}{\partial h_t} = (Y_t - Z_t)V.$$

Output parameters.

$$\frac{\partial L}{\partial V} = \sum_{t=1}^{2}(Y_t - Z_t)h_t, \qquad \frac{\partial L}{\partial c} = \sum_{t=1}^{2}(Y_t - Z_t).$$

Recurrent weight. L_1 sees W via h_1; L_2 via h_2 directly and via $h_1 \to h_2$:

$$\frac{\partial L}{\partial W} = \frac{\partial L_1}{\partial h_1}\frac{\partial h_1}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W},$$

that is,

$$\frac{\partial L}{\partial W} = (Y_1 - Z_1)V(1 - h_1^2)h_0 + (Y_2 - Z_2)V(1 - h_2^2)h_1 + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2)h_0.$$

Input weight and bias [Supplement]. Same three paths, h_{t-1} replaced by x_t and 1:

$$\begin{aligned} \frac{\partial L}{\partial U} &= (Y_1 - Z_1)V(1 - h_1^2)x_1 + (Y_2 - Z_2)V(1 - h_2^2)x_2 + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2)x_1, \\ \frac{\partial L}{\partial b} &= (Y_1 - Z_1)V(1 - h_1^2) + (Y_2 - Z_2)V(1 - h_2^2) + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2). \end{aligned}$$

With $h_0 = 0$ the first and third terms of $\partial L/\partial W$ vanish; the others keep all three.

3.3 General formulation

Input $\tau(X) = (u_1, \dots, u_L)$, $u_\ell \in \mathbb{R}^m$; update $q_{\tilde\theta} : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$, e.g. $q_{\tilde\theta}(h, u) = \phi(Wh + Uu + c)$ with $\tilde\theta = (W, U, c) \in \mathbb{R}^p$:

$$h_0 = \mathbf{0}, \qquad h_\ell = q_{\tilde\theta}(h_{\ell-1}, u_\ell) \quad (\ell = 1, \dots, L), \qquad f_\theta(X) = Ah_L + b,$$

with $A \in \mathbb{R}^{1\times n}$, $b \in \mathbb{R}$, $h_\ell \in \mathbb{R}^n$ and $\theta = (A, b, \tilde\theta)$. For a loss $\mathcal{L}(f_\theta(X), Y)$,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial f}(f_\theta(X), Y)\, \frac{\partial f_\theta(X)}{\partial \theta}, \qquad \frac{\partial f_\theta(X)}{\partial A} = h_L^\top, \quad \frac{\partial f_\theta(X)}{\partial b} = 1, \quad \frac{\partial f_\theta(X)}{\partial \tilde\theta} = A\frac{dh_L}{d\tilde\theta}.$$

$dh_L/d\tilde\theta$ is total; $\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell)$ holds $h_{\ell-1}$ fixed; $J_k = \partial_h q_{\tilde\theta}(h_{k-1}, u_k) \in \mathbb{R}^{n\times n}$.

Theorem 9.4: BPTT formula

For $\ell \le L$,

$$\frac{\partial h_L}{\partial h_\ell} = J_L J_{L-1} \cdots J_{\ell+1}, \qquad \frac{dh_L}{d\tilde\theta} = \sum_{\ell=1}^{L} \frac{\partial h_L}{\partial h_\ell}\, \partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell),$$

(the empty product for $\ell = L$ is the identity). Consequently

$$\frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial \tilde\theta} = \sum_{\ell=1}^{L} \frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial h_\ell}\, \partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell), \qquad \frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L}\frac{\partial h_L}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} J_L \cdots J_{\ell+1}.$$

Proof. Step 1 (state Jacobian). For $k > \ell$, $\partial h_k/\partial h_\ell = J_k\, \partial h_{k-1}/\partial h_\ell$; iterate from I.

Step 2 (one-step recursion). Directly and through $h_{\ell-1}$:

$$\frac{dh_\ell}{d\tilde\theta} = \partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell) + J_\ell \frac{dh_{\ell-1}}{d\tilde\theta}, \qquad \frac{dh_0}{d\tilde\theta} = 0.$$

Step 3 (unrolling). Induction on L; $L = 1$ is Step 2. From $L-1$, Steps 2 and 1 give

$$\frac{dh_L}{d\tilde\theta} = \partial_{\tilde\theta} q_{\tilde\theta}(h_{L-1}, u_L) + \sum_{\ell=1}^{L-1} J_L \frac{\partial h_{L-1}}{\partial h_\ell}\, \partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell) = \sum_{\ell=1}^{L} \frac{\partial h_L}{\partial h_\ell}\, \partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell).$$

Step 4 (loss). $\mathcal{L}$ sees $\tilde\theta$ only through h_L; multiply by $\partial \mathcal{L}/\partial h_L$. ■

The sum over ℓ is the parameter-sharing sum, each copy weighted by its influence on the loss.

4 Vanishing and Exploding Gradients

4.1 RNNs are very deep networks

$\partial h_L/\partial h_\ell = J_L \cdots J_{\ell+1}$ multiplies up to hundreds of Jacobians: factors above 1 explode (**exploding gradients**), below 1 vanish (**vanishing gradients**, the bigger problem). Scalar tanh RNN (Section 3.2): factor $(1 - h_t^2)W$ [Supplement].

Theorem 9.5: a sufficient condition for vanishing gradients [Supplement]

Let $q_{\tilde{\theta}}(h,u) = \tanh(Wh + Uu + c)$ and let $\|\cdot\|$ be the spectral norm. Then

$$\left\| \frac{\partial h_L}{\partial h_\ell} \right\| \leq \|W\|^{L-\ell}.$$

If $\|W\| < 1$ the signal decays geometrically in $L - \ell$.

Proof. Step 1. $J_k = \operatorname{diag}(1 - \tanh^2(a_k))\, W = \operatorname{diag}(1 - h_k^2)\, W$, $a_k = W h_{k-1} + U u_k + c$.

Step 2. Diagonal entries in $[0,1]$: $\|J_k\| \leq \|W\|$.

Step 3. Submultiplicativity and Theorem 9.4: $\|J_L \cdots J_{\ell+1}\| \leq \prod_{k=\ell+1}^{L} \|J_k\| \leq \|W\|^{L-\ell}$. ■

$\|W\| > 1$ need not explode: saturated units ($h_k^2 \approx 1$) shrink the factors. [Supplement]

4.2 Exploding gradients and gradient clipping

When the direction is fine but the length too large, **gradient clipping** with threshold $v > 0$:

$$g \;\leftarrow\; \min\left(1, \frac{v}{\|g\|}\right) g = \begin{cases} g, & \|g\| \leq v, \\ \dfrac{v}{\|g\|}\, g, & \text{otherwise.} \end{cases}$$

same direction, length $\min(\|g\|, v)$ (the slide's max means min). At a cliff an unclipped step jumps away from the valley.

4.3 Vanishing gradients

$\partial \mathcal{L}/\partial h_L$ may be moderate while

$$\frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} \frac{\partial h_L}{\partial h_{L-1}} \frac{\partial h_{L-1}}{\partial h_{L-2}} \cdots \frac{\partial h_{\ell+1}}{\partial h_\ell}$$

is tiny; since

$$\frac{\partial \mathcal{L}}{\partial u_\ell} = \frac{\partial \mathcal{L}}{\partial h_\ell} \frac{\partial h_\ell}{\partial u_\ell},$$

small changes of u_ℓ barely affect $\mathcal{L}$ (loosely, u_ℓ is forgotten; large changes may still matter), and the model cannot learn what to keep at time ℓ.

5 Long Short-Term Memory

Motivation. An additively updated, gated memory cell lets gradients cross long time spans.

5.1 The LSTM cell

Vanilla: $h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$. The LSTM adds a gated **cell state** c_t (long memory).

Definition 9.6: LSTM

With σ the logistic sigmoid and $\odot$ elementwise multiplication,

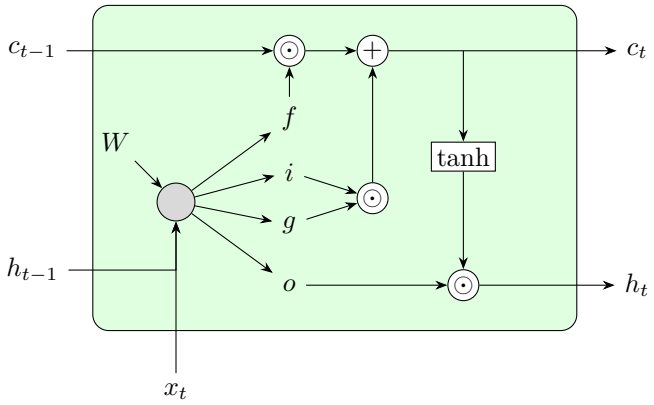
$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), & f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), & g_t &= \tanh(W_g x_t + U_g h_{t-1} + b_g), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

- Input gate i_t: how much of g_t to write.
- Forget gate f_t: how much of c_{t-1} to keep.
- Output gate o_t: how much of the cell to reveal in h_t.
- Candidate ("gate gate") g_t: new content.

Compactly,

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix},$$

W is $4h \times 2h$ ($4h \times (h+m)$ for input dimension m), output split into four blocks. Other notation: $[h_{t-1}, x_t]$, $\tilde{C}_t = g_t$, $* = \odot$.



The LSTM cell: W acts on the stacked (h_{t-1}, x_t); the cell state runs along the top.

5.2 Gradient flow through the cell

Theorem 9.7: the cell path

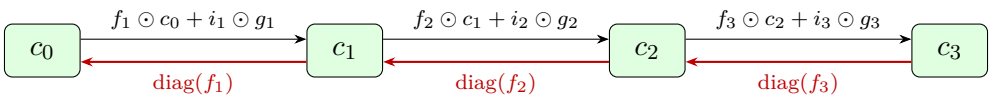
Holding the gates fixed, $\dfrac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t)$. Along the path that goes only through cell states,

$$\frac{\partial c_L}{\partial c_\ell} = \operatorname{diag}\Big(\prod_{k=\ell+1}^{L} f_k\Big) \qquad \text{(product taken entrywise)}.$$

Proof. Step 1. In $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ only the first term contains c_{t-1}, as $\operatorname{diag}(f_t) c_{t-1}$.

Step 2. Chain the diagonal Jacobians. ∎

Vanilla factors $\operatorname{diag}(1 - h_t^2)W$ repeat W and can only shrink; the LSTM factor is the learned forget gate. With $f_t \approx 1$ the gradient passes almost unchanged ("uninterrupted gradient flow"); $f = 1$, $i = 0$ preserves the cell indefinitely.



Backward (red) along the cell states multiplies only by forget gates.

Remark 9.8: what the LSTM does not guarantee

Other paths (through h_{t-1} and gates) remain and f_t may be far from 1: no guarantee, just an easier route to long-range dependencies.

6 Applications

Language modeling and image captioning; also SIRD epidemic ODEs, where an LSTM learns the time-varying infection rate α.

6.1 Language modeling

Generation, translation and summarization all model token sequences with f_θ.

Definition 9.9: one-hot encoding (tokenization)

Token t of an n-token vocabulary is $\mathbf{x}_t \in \mathbb{R}^n$ with a single 1 at its index.

Definition 9.10: RNN language model

Sentence generation is next-token prediction. At step t:

$$\begin{aligned} \mathbf{z}_t &= W_e \mathbf{x}_t + \mathbf{b}_e, & W_e &\in \mathbb{R}^{d\times n}, \\ \mathbf{h}_t &= \mathrm{LSTM}(\mathbf{z}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}; \theta), & \mathbf{h}_t &\in \mathbb{R}^d, \\ \hat{\mathbf{x}}_{t+1} &= W_d \mathbf{h}_t + \mathbf{b}_d, & W_d &\in \mathbb{R}^{n\times d}. \end{aligned}$$

(word embedding, recurrent state, vocabulary scores), from $\mathbf{x}_0 =$ `<SOS>` to `<EOS>`.

Training. Read `<SOS>`, $\mathbf{x}_1, \ldots, \mathbf{x}_{T-1}$, predict the next token, with word-level loss

$$\sum_{t=1}^{T} \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t), \qquad \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t) = -\log \mathrm{softmax}(\hat{\mathbf{x}}_t)_{k_t},$$

(k_t the true index). Since

$$p_\theta(\mathbf{x}_1, \ldots, \mathbf{x}_T) = \prod_{t=1}^{T} p_\theta(\mathbf{x}_t \mid \mathbf{x}_0, \ldots, \mathbf{x}_{t-1}).$$

this is maximum likelihood (Chapter 2).

Inference. From scores $S(\mathbf{x}_t)$:

- Greedy (deterministic): $\mathbf{x}_t = \arg\max S(\mathbf{x}_t)$.
- Probabilistic (stochastic): sample $\mathbf{x}_t \sim S(\mathbf{x}_t)$ from the softmax probabilities.

Feed back until `<EOS>`. Greedy always gives the same sentence; sampling varies.

Machine translation is the delayed many-to-many pattern.

6.2 Image captioning

Describe image $\mathbf{x}$ by sentence $\mathbf{y}$:

1. Recognize the visual patterns in the image with a CNN g_ϕ (Chapter 7).
2. Generate a sentence conditioned on the image with an RNN language model.

The model is

$$f_\theta(\mathbf{y} \mid g_\phi(\mathbf{x})) :$$

features $g_\phi(\mathbf{x})$ condition the language model ("man in black shirt is playing guitar"). Variants: naive, and with attention (Chapter 10).

Naive captioning. The last FC feature of an ImageNet CNN (4096 VGG, 2048 ResNet; $224 \times 224 \times 3$ input) is linearly mapped to the embedding dimension and fed as the first word, then `<sos>`, then generation as in Definition 9.10. Train on (image, sentence) pairs with word-level cross-entropy.

Example 9.11: the captioning loss (lecture example)

$\mathbf{y}_t$ one-hot truth, $\hat{\mathbf{y}}_t$ predicted distribution. Vocabulary {man, plays, guitar, `<EOS>`, `<SOS>`}, target "man plays guitar": inputs `<SOS>`, man, plays, guitar, targets $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3, \mathbf{e}_4 \in \mathbb{R}^5$. If $\hat{\mathbf{y}}_3 = (0.05, 0.1, 0.8, 0.04, 0.01)$, then

$$\mathcal{L}_3 = -\sum_{k=1}^{5} y_{3,k} \log \hat{y}_{3,k} = -\log 0.8 \approx 0.223,$$

and $\mathcal{L} = \sum_{k=1}^{4} \mathcal{L}_k$.

A practical issue. Previous words are at dependency distance 1, the image only at step one, so a strong language model may ignore the image. Remedies:

- **Shortcuts.** Feed the image feature at every step (distance 1, but still ignorable).
- **Attention.** Different regions for different words (Chapter 10); uniform weights recover the global feature.